Lessons 2 and 3

ILAI (M1) @ LAAI I.C. @ LM AI

18 September 2025

Michael Lodi

Department of Computer Science and Engineering

These slides draw very heavily from Simone Martini's slides.

One-slide Recap of L1

Computations are performed by (abstract) machines. They are a sequence of elementary operations that the machine can perform. Computations are described in a language (the language the machine can execute – in our case, the Python machine).

Elementary operations acts on values/objects, organized in types (set of values, their presentation, the elementary operations, the name of the type)

- integers (`int`, no overflow), strings (`str`, immutable), subset of rationals (`float`)

Expression: a phrase for obtaining a value we are interested in.

Evaluated from left to right.

- elementary operations on numbers/strings/bools, `input()` (transfer to internal state, gives a `str`)

Command: a phrase we use to change the state of the machine, not interested in value

- assignment (evaluate right side first, change the internal states i.e. names assoc values); `print()` (transfer to the external state)

Program: plain text, each line a single command/expression, need to be run

Pending questions

Are Python floats double?

- It depends on the implementation. The most common, CPython implementation, yes (they use the C double).

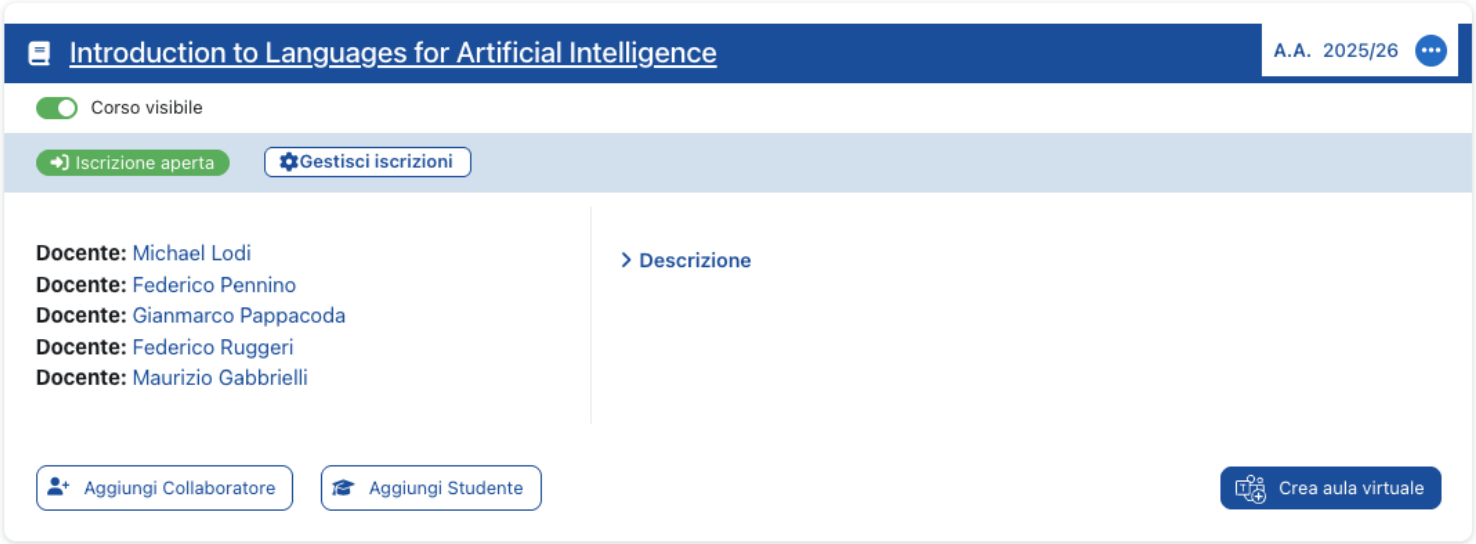
When to use explicit float( ) cast?

- Mainly when parsing strings. Otherwise use things like 1. or 5/2

Is there an analogous of Java's StringBuilder?

- No. Python strings are immutable.
- But can play around with strings in many ways, e.g.
    - https://docs.python.org/3/library/stdtypes.html#str.join
    - https://docs.python.org/3/tutorial/inputoutput.html

Remember to join Virtuale!



… and to do the exercises I post ☺

Conditional command

It is a *compound* command:

it is composed (also) from other commands

Its use: altering the sequential order of execution of a program, according to some *conditions*

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Conditional command: first form

```
if <condition>:                    guard
     <t-block>                     then branch
else:
     <e-block>                     else branch
```

Syntax:

`if` and `else` are *reserved names*

A reserved name (or reserved word) has the structure of a standard name but *cannot be used as a name* because of its special role inside the language

Conditional command: first form

```
if <condition>:                    guard
     <t-block>                     then branch
else:
     <e-block>                     else branch
```

semantics:

We evaluate the guard. It the guard is true, we execute the “then” branch; if the guard is false, we execute the ”else” branch

After the then/else branch (only one of those!) the execution proceeds from what follows the conditional command

Conditions: boolean values

`if` <*condition*>`:` guard

What is this <*condition*> `?`

Syntactically:

any expression of type `bool`

Data types: bool

The type of the truth values:

- *values* : only two, *true* and *false*
- *presented* : True, False
- *elementary operations* : and (logical conjunction), or (logical disjunction), not (negation). They are defined by the usual truth tables
- *name* : bool

- Exp1 `and` Exp2: `True` iff both Exp1 and Exp2 are True
- Exp1 `or` Exp2: `False` iff bothExp1 and Exp2 are False

Logical operators

`not` (negation):

transforms `True` into `False` and `False` into `True`

`not True` has value `False`

`not False` has value `True`

`and` (conjunction: et):

<exp1> `and` *<exp2>* has value `True` iff
both *<exp1>* and *<exp2>* have value `True`

`or` (disjunction: vel):

<exp1> `or` *<exp2>* has value `False` iff
both *<exp1> and <exp2>* have value `False`

Comparison operators: producing bool values

less than:	`<`
less than or equal :	`<=`
greater than:	`>`
greater than or equal:	`>=`
Equal:	`==`
Non equal:	`!=`

Lazy evaluation of logical operators

Python expressions are

completely evaluated from left to right:

```
7*0*(int(input()))
```

the `input()`

is always performed, even if the result (0) is already known

Lazy evaluation of logical operators

Python expressions are

completely evaluated from left to right:

```
7*0*(int(input()))
```

the `input()`

is always performed, even if the result (0) is already known

Boolean expressions are an exception to this

lazy evaluation of Booleans: as soon as we find a value that allows to know the final result, the evaluation terminates

```
(0==0) and (0==1) and (0==int(input()))
```

not evaluated

Nested if

Write a program which input an int x and an int y and prints:

0 if x is 0

y/x if y is negative

-y/x if y is positive

100 if y is zero

nested "if"s

nested if

0 if x is 0
y/x if y negative
-y/x if y positive
100 if y is 0

```
x = int(input("Provide x: "))
y = int(input("Provide y: "))
if x == 0:
    print(0)
else:
    if y < 0:
        print(y/x)
    else:
        if y > 0:
            print(-y/x)
        else: #y is 0
            print(100)
```

nested if

0 if x is 0
y/x if y negative
-y/x if y positive
100 if y is 0

```
x = int(input("Provide x: "))
y = int(input("Provide y: "))
if x == 0:
    print(0)
else:
    if y < 0:
        print(y/x)
    else:
        if y > 0:
            print(-y/x)
        else: #y is 0
            print(100)
```

```
x = int(input("Provide x: "))
y = int(input("Provide y: "))
if x == 0:
    print(0)
elif y < 0:
    print(y/x)
elif y > 0:
    print(-y/x)
else: #y is 0
    print(100)
```

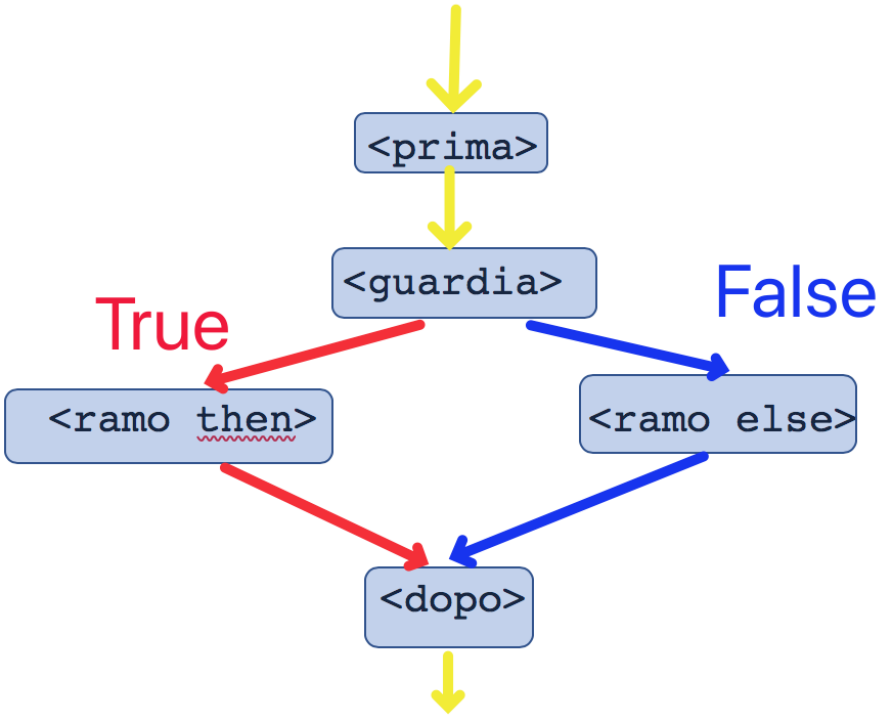
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

General form of the conditional

```
if <expr1 bool>:
    <block1>
elif <expr2 bool>:                    optional
    <block2>
…
elif <exprk bool>:                    optional
    <block>
else:                                 optional
   <block-e>
```

Review: conditional command

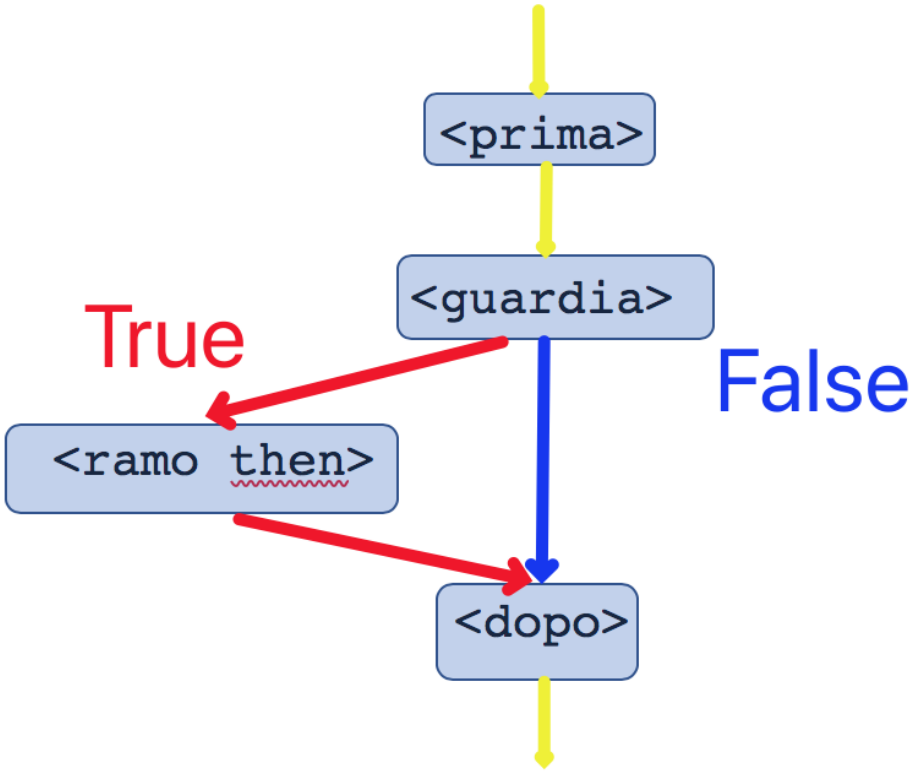
```
<prima>
if <guard>:
    <then branch>
else:
    <else branch>
<dopo>
```



Review: conditional command

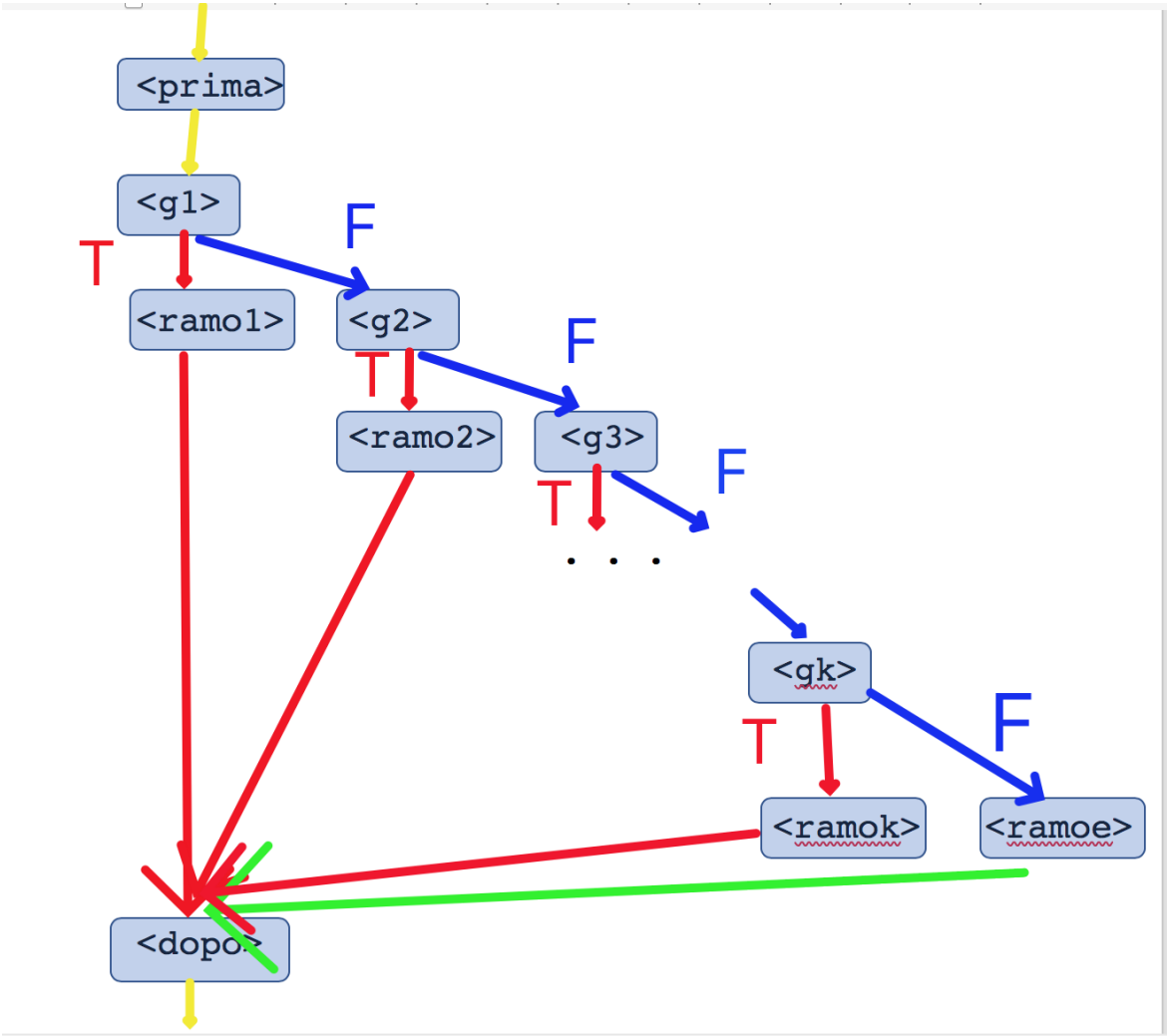
else is optional

```
<prima>
if <guard>:
      <then branch>
<dopo>
```



Review: conditional command

```
<prima>
if <g1>:
     <ramo1>
elif <g2>:
     <ramo2>
…
elif <gk>:
     <ramok>
else:
     <ramoe>
<dopo>
```



Blocks

Block:

- a sequence of lines
- each line with a single command (assignment, print, if…)
- all of them at the same *indentation*

  (same distance from the left margin)

It is used to «*structure*» the execution
in with compound commands (e.g., `if`)

Other programming languages use parenthesis `{,}`.
Python uses indentation

No local scopes in blocks (but yes in functions or classes)

scope: portion of program (execution) where that name is visible

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

QUIZ TIME!

Always the same
link / code / QR code

https://app.wooclap.com/ILAI25

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Review: blocks. example

How many blocks?

What does it print on input 20?

```
x=int(input())
if x!=0:
    y=10
    print(y)
else:
    z=20
    print(z)
x=100
print(x)
```

Review: blocks. example

How many blocks?

```
x=int(input())
if x!=0:
    y=10
    print(y)
else:
    z=20
    print(z)
x=100
print(x)
```

Review: blocks. another example

What does it print on input 20? And on input 0?

```
x=int(input())
if x!=0:
    y=10
    print(y)
else:
    z=20
    print(z)
    x=100
print(x)
```

One-slide Recap of L1, updated

Computations are performed by (abstract) machines. They are a sequence of elementary operations that the machine can perform. Computations are described in a language (the language the machine can execute – in our case, the Python machine).

Elementary operations acts on values/objects, organized in types (set of values, their presentation, the elementary operations, the name of the type)

- integers (`int`, no overflow), strings (`str`, immutable), subset of rationals (`float`), **truth values (`bool`, presented with capital, lazy eval of `and`/`or`)**

Expression: a phrase for obtaining a value we are interested in.

Evaluated from left to right.

- elementary operations on numbers/strings/bools, `input()` (transfer to internal state, gives a `str`)

Command: a phrase we use to change the state of the machine, not interested in value

- assignment (evaluate right side first, change the internal states i.e. names assoc values); `print()` (transfer to the external state), **`if`**, **`if-else`**, **`if-elif-else`**

Program: plain text, each line a single command/expression, need to be run

Structuring a program: functions

A *function* (in the context of programming languages) is a *named sequence of commands*, which performs a computation

Functions help to structure a program in "*subprograms*", each of which performs a simpler task

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

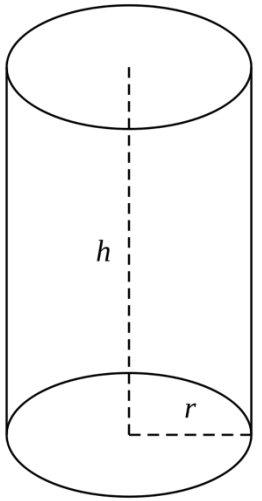
Structuring a program: functions

Compute the volume of a cylinder:

area of its basis circle

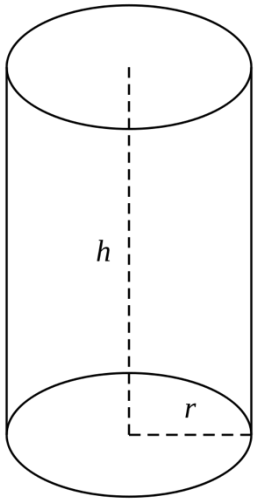
*

the height of the cylinder



Structuring a program: functions

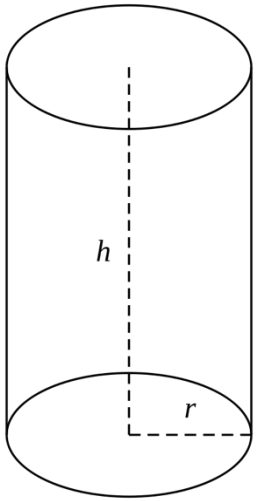
```
def cylinder_vol(r,h):
    pi=3.1415
    return (pi*r**2)*h

print(cylinder_vol(2,3))
print(cylinder_vol(6,7))
```



Structuring a program: functions

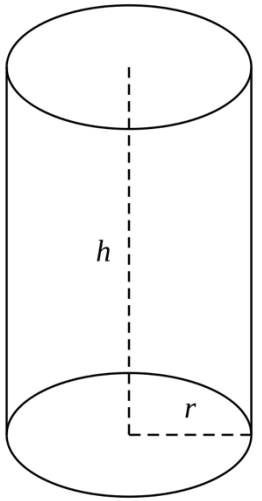
```
def cylinder_vol(r,h):
    pi=3.1415
    return (pi*r**2)*h
```

However, we may split the problem in two subproblems (*of course, this is a toy example!*)



Structuring a program: functions

```
def area_circle(r):
    pi=3.1415
    return pi*r**2

def cylinder_vol(r,h):
    return area_circle(r)*h

print(cylinder_vol(2,3))
print(cylinder_vol(6,7))
```



Functions: definitions vs use

1. Functions should be *defined*

```
def cylinder_vol(r,h):
    return area_circle(r)*h
```

A binding between the name and the *body* is inserted into the internal state. *No computation is done*

2. Functions may be *used* (*called*):

```
print(cylinder_vol(2+1,3*2))
```

arguments are bound to the parameters, and the body *is executed.* A value is *returned.*

Functions: definition

```
def <name> (<formal parameters>):
    <block>
```

header

body

`def` is a reserved name

It has the structure of a standard name but *cannot be used as a name* because of its special role in the header of a function.
Other reserved names we met: `if`, `and`, `or`, `not`, …

<formal parameters> : comma separated ***names***
(can be empty – but still needs parentheses)

<block> : is a sequence of commands all at the same indentation

A ”def” is a compound command

Functions: use

To use (call) a function we use the syntax:

<name>(<list of arguments>)

which is an *expression*

<list of arguments> : a comma separated list of **expressions** (**in the same number** of the formal parameters in the header of the function)

(can be empty – but still needs parentheses)

Also known as: *actual parameters*

return

`return` <expression>

is a command that may appear *only* inside the body of a function

When executed, *it forces the termination* of the execution of the body; the value of <expression> is “returned” as the value of the function call

`return` is a reserved name

Functions: semantics

```
def f(x):
    body
```

A def of function introduces into the internal state a binding between the name of the function and its body

`f(exp)`

When a function is called:

`exp` is evaluated to a value `v`

in the internal state (*in the local frame for* `f`),
`x` (the formal parameter of f) *is bound to*
`v` (the value of the actual parameter)

`body` is executed in this state

the evaluation of any `return <exp2>` causes the termination of the function.
The value of `<exp2>` is "returned" as value of the call

Functions: *formal* and *actual* parameters

Formal parameters (or *parameters*, for short) are *names*.
Comma separated if more than one.
If there are no parameters, then the def has the form

```
def f():
    body
```

Actual parameters (or *arguments*, for short) are *expressions*.
Same number of the actual parameters.
At the moment of the call they are bound to the formal parameters, respecting their order

Functions: the evolution of the state

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Functions and the internal state

Any call to a function creates a new frame in the internal state

When a function "returns" its frame is erased from the state

Frames are added and removed from the state like dishes from a stack of dishes. Hence the name for the internal state: the *frame stack*, or the *call stack*

Formal parameters and any name occurring to the left of an assignment in the body of a function is *local* to that function:

- the binding is created inside the frame for that function
- the binding is deleted together with the frame, at return time

Example: frames on the stack

Check on pythontutor.com:

```
def perimeter(base,h):
    P=(base+h)*2
    return P
    return 100   #never executed! (this is a comment)
def z(h):
    return 0
def f(x):
    N=z(1)+1+x
    return N+1

per=0
per=perimeter(10,2)
w=f(9)
print(per,w)
```

Local names

Check on pythontutor.com:

```
A=10
def foo(x):
    return A+x
def fie(y):
    A=100
    return A+y

print(foo(10))      # prints 20
print(fie(10))      # prints 210
print(A)            # prints 10
```

Functions without return

What if execution of (the body of) a function terminates without a return?

```
def triple(x):
    x = abs(x) #pre-defined function
    print('three times x is: ', 3*x)


triple(3)
```

Functions without return

If the execution of a function reaches the end of its body, an implicit `return` is inserted by the machine

```
def triple(x):
    x = abs(x) #pre-defined function abs()
    print('three times x is: ', 3*x)
    return     #this is not necessary

triple(3)
```

Functions without return: the value None

Let's `print` the value returned by such a function:

```
def triple(x):
    x = abs(x) #pre-defined function abs()
    print('three times x is: ', 3*x)
    return     #this is not necessary

print(triple(3))    #not a very good idea
```

Functions without return: the value None

`None` is a special value (of type `NoneType`)

It is the value of those expressions for which

we are interested in the action they have on the state

because their value is irrelevant

`return`

is just an abbreviation of

`return None`

NoneType

The type of None:

- *values* : a single element
- *presented* : `None`
- *elementary operations* : no operations
- *name* : `NoneType` (it cannot be used as a converter)

None

Try in Python:

```
print(print(10))
```

Be sure you understand what happens!

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

QUIZ TIME!

Always the same
link / code / QR code

https://app.wooclap.com/ILAI25

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Default and keyword parameters in functions

The predefined function `print` has optional parameters, to be passed *by keyword*

```
print(10, 20, sep=' *** ')          #default for end is '\n'
print(30, 40, end=' ===== ')        #default for sep is ' '
print(50, 60, end='\n\n^^\n')
print(70, 80)
```

prints

```
10 *** 20
30 40 ===== 50 60

^^
70 80
```

Default and keyword parameters in functions

Arguments

by keywords and

with *default*

can be used *with user-defined functions*

more on this in a following lesson

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Predefined functions

The Python machine has many pre-defined functions

We know some: `len()`, `int()`, `float()`, `input()`, etc.
Some others: `max()`, `min()`, `abs()`, etc.

See the documentation:
https://docs.python.org/3/library/functions.html

Predefined functions: Libraries (modules)

The Python machine has many pre-defined functions

We know some: `len()`, `int()`, `float()`, `input()`, etc.
Some others: `max()`, `min()`, `abs()`, etc.

See the documentation:
https://docs.python.org/3/library/functions.html

Also a large collection of *libraries* (*modules*)
that is additional collections of functions, and,
more generally, pre-defined names

Libraries (modules)

Some available libraries

math : `sin()`, `cos()`, `pi`, `factorial()`, `gcd()`, etc.

random : `randint(a,b)`, `random()`, etc.

string : `punctuation`, `digits`, `ascii_uppercase`, etc.

The entire catalogue:

https://docs.python.org/3/library/

Importing *names* from a module/library

Command: `from`

```
from <module> import <name-list>
```

Example:

```
from math import sqrt, e
```

Importing all names from a module:

```
from <module> import *
```

Importing *names* from a module/library

Command: `import`

```
import <module-name>
```

All fully qualified names of `<module-name>` are available

Example:

```
import math
root=math.sqrt(2)**math.e
```

or:

```
import math as m
root=m.sqrt(2)**m.e
```

Many more libraries

User-supplied libraries:

the Python package index

https://pypi.org/

In Thonny: Tools -> Manage packages

Or with pip (pip3) command line tool

Some of these:

`Matplotlib` : visualization, graphs, plots

`NumPy` : numerical computations, arrays

`Pandas` : data analysis

`...`

Problem

Given the string ‘LODI’ we want to print its elements one by one, on a new line

```
s='LODI'
print(s[0])
print(s[1])
print(s[2])
print(s[3])
```

Another problem

Given a string from user input, we want to print its elements one by one, on a new line

```
s=input()
???????
```

bounded iteration (definite iteration): for

```
for <name> in <sequence>:
    <block>
```

compound command (like `if`)

sequences: for the moment only strings

bounded iteration: for

```
s='COOP'
for c in s:
    print(c)
```

Express a repetion: in this case the command

```
print(c)
```

is executed on all elements (all characters) of the (value of) s

A linear scan

Task: print the non blank elements (i.e, `!=' '`) of a string `st`

```
st='bologna is a nice town'
for el in st:
    if el!=' ':
        print(el)
```

Example of a pervasive programming pattern:

linear scan of a sequence

For: a more complete view

Semantics:

```
for <name> in <sequence>:
    <block>
```

(0) <sequence> *is computed (it is an expression) and "frozen"*

(1) *If <name> does not exists, it is created*

(2) <name> is bound to the *first* element of the <sequence>

(3) Evaluate <block>

(4.1) If there is a *next* element in <sequence>, bind <name> to this element; repeat from (3)

(4.2) If there is *not* a *next* element in <sequence>, the evaluation of the command terminates (and hence the execution passes *after* the for)

QUIZ TIME!

Always the same
link / code / QR code

https://app.wooclap.com/ILAI25

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Edge cases: check them

```
c = 42
for c in 'lodi':
    print('in:', c)
print(c)
```

```
s = 'lodi'
for c in s:
    s = 'Michael'
    print(c)
print(s)
```

```
c = 42
for c in '':
    print(c)
print(c)
```

The boolean operator in

```
<element> in <sequence>
```

a boolean expression with value `True` iff
<element> is an element of <sequence>

Examples:

```
'p' in 'michael'      has value False
'a' in 'aeiou'        has value True
'A' in 'aeiou'        has value False
```

Of course both <element> and <sequence> may be arbitrary expressions

Only if <sequence> is a string:

True if <element> *is a substring of <sequence>*

```
'ae' in 'michael'     has value True
```

operator in

Do not confuse the two uses of the reserved name `in`:

```
for <name> in <sequence>:
```

is the header of an iteration

```
<element> in <sequence>
```

is a boolean expression

We cannot mix the two. Example:

```
for s in 'michael' and s in 'lodi':  NO! WRONG!
```

It is not correct Python.

We may combine logical expressions:

```
if s in 'michael' and s in 'lodi':
```

One-slide recap so far

Function: sequence of commands with a name. Returns a value.

Defined with `def`, name, parenthesis, formal parameters (names)

Called with name and actual parameters (*arguments*, are expressions)

Value of actual parameters bound to the formal parameters, which are local names

Also any name occurring to the left of an assignment is local to the function

Function without return, return the value `None` (of type `NoneType`): a value of expressions for which the value is irrelevant (we are interested in «side effects», actions on the internal state).

We can import names (and functions) from libraries with `import` or `from … import`

We can use `for <name> in <sequence>`: to scan every element of a sequence. `<name>` is just a name like all the others (no local scope).

The «`in`» in the `for` should not be confused with the boolean operator `in`, which returns `True` if an element (for strings only, also substrings) appear in a sequence.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Objects

An object is a value envelopped in an identity

This identity is unique during a run of the Python machine

In Python, *any value is an object* (even int values)

We may have distinct objects (with different identities) and same value

Identity of an object is completely controlled by the Python machine:

we may inspect the identity of an object : function

`id( )`

but we cannot change it

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Objects

An object is a value envelopped in an identity
This identity is unique during a run of the Python machine

`==` is equality between *values*

(nothing to do with identities)

`is` : comparison operator

`True` iff identities are the same

Obvious: if two objects are identical on `is`, they are also `==`

example: isequal.py

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Objects: identity

- At this stage of the course, the `is` relation is not much relevant
- it will become relevant when *mutable* objects will be introduced

Important:

we can never assume identity of objects from the fact that they have the same value

On the contrary *we must always assume* that

any operation creates new objects

For: ``freezing'' the sequence

```
for <name> in <sequence>:
    <block>
```

(0) <sequence> *is computed and "frozen"*

What is frozen is the object (its identity). Therefore in:

```
s='Lodi'
for c in s:
    print(c)
    s='zzzz'
```

the assignment `s='zzzzz'` happens,
but it has no effect on the repetition,
because the object `'zzzz'` is not the one controlling the for

What will this program print?

Try on your own (without running it)

```
a = "Lodi"
for c in a:
    print(c)
    c = 'z'
    a = a + c
    print(c, a)
```

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Objects: methods

The envelope enclosing an object contains, besides its identity, also other information

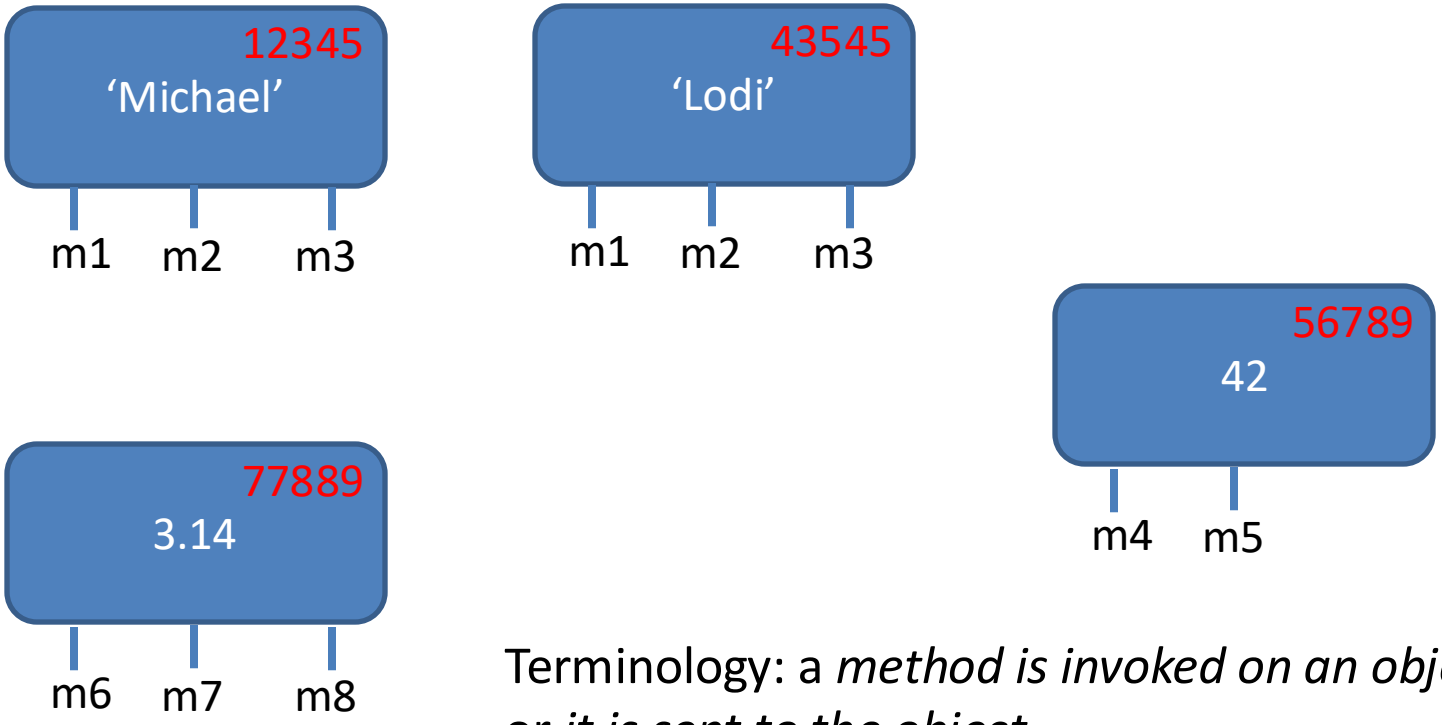
An important information of an object are *its methods*

An object is an "active value":
we may stimulate the object through the *methods* offered by its envelope

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Objects: methods

Objects of the same *type* share the same methods



Terminology: a *method is invoked on an object*
or *it is sent to the object*
or also (I like this less) it is *called on* the object

Objects: methods

Methods act on the object on which they are invoked; they stimulate the object to do something (e.g., to return a value)

Syntax to invoke a method:

<object>.<method>(<optional arguments>)

Examples:

str offer the method upper()

```
s='bologna BO'
t=s.upper()
print(t)
```

Objects: methods

Methods act on the object on which they are invoked; they stimulate the object to do something (e.g., to return a value)

examples in Thonny (file metods_shell.txt)

Recall: the object receiving the method can be a constant, a name, an expression… (in summary: can be any Python expression evaluating to an object)

Some methods offered by str

See them at:

https://docs.python.org/3/library/stdtypes.html#string-methods

`str.capitalize()`

Return *a copy of the string* with its first character capitalized and the rest lowercased.

`str.isalpha()`

Return True if all characters in the string are alphabetic and there is at least one character, False otherwise.

`str.isdigit()`

Return True if all characters in the string are digits and there is at least one character, False otherwise.

Some methods offered by str

`str.lower()`

Return a copy of the string with all the cased characters converted to lowercase.

Methods with parameters (observe the notation used by the documentation) :

`str.find(sub[, start[, end]])`

Return the lowest index in the string where substring *sub* is found within the slice s[start:end]. Optional arguments *start* and *end* are interpreted as in slice notation. Return -1 if *sub* is not found.

`str.count(sub[, start[, end]])`

Return the number of non-overlapping occurrences of substring *sub* in the range [*start*, *end*]. Optional arguments *start* and *end* are interpreted as in slice notation.

Methods offered by float

`fl.as_integer_ratio()`

Return a pair of integers whose ratio is exactly equal to the original float and with a positive denominator

`fl.is_integer()`

Return True if the float instance is finite with integral value, and False otherwise:

Some methods offered by sequences, hence by str and other sequence types we will see

`s.index(x)`

Return the index of the first occurrence of *x* in *s;*

it is an error if x is not an element of s

`s.count(x)`

total number of occurrences of x in s

A new type: tuple

- `int`, `float`, `bool`, `str`, `NoneType`

  are simple types

- tuples are a structured type (compound type):

  its values are groups of values

  each of them of arbitrary type

- tuples are *immutable sequences* of objects
  - (strings are also immutable sequences)

tuple

Values: finite sequences of values
each of them of arbitrary type

Presentation: `(<object1>, … , <objectk>)`

or also without parentheses () :
the comma `,` is the constructor for tuples

`()` is the empty tuple

the tuple with a single element is `(<element>,)`

Operations: concatenation (`+`), repetition (`*`), length `len()`,
selection `[…]`

They are the common operations on most other types of sequences (eg, strings)

tuple

Examples:

```
(10.3, 100, 'simone')
(3,)
()
(1, 2, (100,200), 3)
(10, 20,)
10,20
```

Non examples:

```
(3)
(10,,20)   {10,20}    [10,20]    (,0)
```

Indexes and scan: like for str

Indexes on tuples: similar to `str`

- they start from 0
- negative indexes: count from the end

We may perform a

`for`

on a tuple

```
for e in (1,2,'bologna',(3,4)):
    print(e)
```

Operations create new objects

```
T = (1,2,3)
T = T + (4,5)
```

Any operation creates a new object!

On the contrary…

```
V = (100,200)
S = V
```

`S` and `V` are two names for the same object

tuple() as type converter

As any name of a type, `tuple()` may be used to convert values into tuples

```
tuple()              ()
tuple('bolo')        ('b','o','l','o')
tuple((1,2,3))       (1,2,3)

tuple(100)           error

tuple(range(2,6))    (2,3,4,5)
```

Select and "name" elements

```
T = (1,2,(100,200),3)
X = (10,20)
W = (9,X,11)
Z = T[2]
```

➔try them in Pythontutor

aliasing is possible: different names (the aliases) refer to the same object

Aliasing on tuple is harmless, because tuple are immutable

Generalised assignment

Standard assignment:

```
<name>=<expression>
```

Generalised:

```
<name1>,…,<namek>=<sequence with exactly k elements>
```

Recall how an assignment is evaluated!

1. Evaluate the RHS, and obtain its value
2. Bind this value to the name(s) in the LHS

Examples

```
A,B = (1,2)
C, D, E = A+1, 5, B
C, E = C+1, E+1
```

Same number of elements on the two sides !

Examples

```
A,B = (1,2)
C, D, E = A+1, 5, B
C, E = C+1, E+1
```

Same number of elements on the two sides !

```
A,B = B,A
```

this the “Pythonic” way of swapping two bindings

Selecting portions of sequences: slice

A *slice* is a *subsequence*

At first sight: a generalization of "selection of one element"

```
s='Bologna b school'
print(s[0])             # 'B'
print(s[8:11])          # 'b s'
print(s[:7])            # 'Bologna'
print(s[10:])           # 'school'
```

Slice

<*sequence*>[<*start*>:<*end*>]

Semantics:

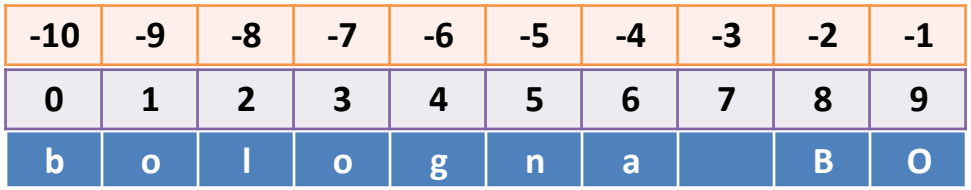
subsequence of <*sequence*> starting at <*start*> index (included) up to <*end*> index (*non included*)

Warning: <*start*> < <*end*>

otherwise the sequence is empy

Slice

Examples:



-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7	8	9
b	o	l	o	g	n	a		B	O

```
s='bologna BO'
```

Who are they?

```
s[0:4]    bolo
s[3:-1]   ogna B
s[3:2]
s[3:4]      o
s[3:3]
```

Slice

Examples:

-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7	8	9
b	o	l	o	g	n	a		B	O

s[: <end>] starts at 0
s[<start> :] ends at len(S)+1, last element included

```
s='bologna BO'
```

Who are they?

```
s[:4]
s[3:]
s[:]
s[:-1]
```

Slice: is *not* a *generalised selection*

Warning:

a slice is a «window» on the sequence

-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7	8	9
b	o	l	o	g	n	a		B	O

```
s='bologna BO'
```

what is `s[6:12]` ?

Slice: step

<*sequence*>[<*first_index*> : <*last_index*> : <*step*>]

Semantics:
subsequence starting at <first_index>,
terminating at <last_index>-1,
each element <step> elements distant from the preceding one

<*sequence*>[<*first_index*>: <*last_index*>]
is shorthand for
<*sequence*>[<*first_index*>: <*last_index*> : 1]

Slice: step

subsequence starting at <first_index>, terminating at <last_index>-1, each element <step> elements distant from the preceding one

-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7	8	9
b	o	l	o	g	n	a		B	O

```
s[1:9:2]
s[ : 100 : 3]
s[3 : : 3]
s[3 :-1 : 3]
```

Slice: negative step

When the step is negative, the subsequence is extracted
still from <start> included to <end> excluded
from the end; in this case *<start>* > *<end>*
so it is extracted backwards

-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
0	1	2	3	4	5	6	7	8	9
b	o	l	o	g	n	a		B	O

```
s[5:2:-1]    ngo
s[ : 3:-1]
s[6::-1]
s[6:0:-1]
s[ : : -1]
s[8:2:-3]
```

warning: `s[8:-1:-1]`

from end of the sequence (last included!)
to start (NB: 0 included!)
(NB: 0 (<end>) excluded!)
(copy of) s reversed

Operations create new objects

On integers is obvious:

x=8

x=x+1

In the second assignment, we RHS evaluates to 9, which is bound to name x

9 is a new value, is not (obviously!) a modification of 8

(which does not make any sense)

The same true for other types!

And then it appears less is obvious…

Operations create new objects

On tuples:

x=(10,20,30)

y = x

x= x + (40,)

In the second assignment, we RHS evaluates to (10,20,30,40) which is bound to name x

(10,20,30,40) is a new value, is not a modification of (10,20,30)

(which does not make any sense… because tuples are immutable!)

operations create new objects

Slices, like any other operation, create new objects!

```
S = (10,20,30)
W = S[:]
V = S
```

`S` and `V` are *two names* for the *same object* (they are *aliases*)

`W` is bound to a new object, which has the same value of `S`

Linear scanning of a sequence: backwards

```
S='bologna'
for c in S[::-1]:
    print(c)
```

Exercises

Write on the whiteboard «Strings and tuples are immutable» 1000 times

Write a funciton taking as argument a sequence S and returning True if and only if there are two consecutive equal elements

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Exercises

Write on the whiteboard «Strings and tuples are immutable» 1000 times

- I need a «repeat 1000 times»

Write a funciton taking as argument a sequence S and returning True if and only if there are two consecutive equal elements

- I need to access and manipulate indexes of S

Special sequences: range

`range(<start>,<end>,<step>)`

sequence of integers between <start> and <end>-1, each <step> apart from the previous

*+ and * are not defined on ranges*

`range(<end>)` shorthand for `range(0,<end>,1)`

If `<end> < <start>` range is empty

negative step: same as for slices

Special sequences: range

`range(<start>,<end>,<step>)`

sequence of integers between <start> and <end>-1, each <step> apart from the previous

We may perform a `for` *on a range!*

Exercises

Write on the whiteboard «Strings and tuples are immutable» 1000 times

- I need a «repeat 1000 times»

Write a funciton taking as argument a sequence S and returnit True if and only if there are two consecutive equal elements

- I need to access and manipulate indexes of Python

introrange.py

Using range on other sequences

Range allows to access the indexes of a sequence

Given a tuple `T`,

count the number of *identical contiguous pairs*

Ex: on `(1,2,2,3,4,4,4,5)` return `3`

Example with range

Given a tuple `T`,

count the number of *identical contiguous pairs*

```
def countpairs(T):
    res=0
    for i in range(len(T)-1): #end one index before
        if T[i]==T[i+1]:
            res = res +1
    return res
```

Exercises on Virtuale - sequences

More structured stuff

- Slides with details to study! (e.g. string quotes)
- Exercises

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Exercises on Virtuale - sequences

More structured stuff

- Slides with details to study! (e.g. string quotes)
- Exercises

Current Python and Numpy version on Virtuale

(as of 18 sept 2025 – updates out of our control)

Python version: 3.10.12 [GCC 11.4.0]

NumPy version: 1.26.4

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA